

Advanced Programming in Python Project Report

Tamim Hussein
2502572

1 Overview

This project implements a DNA sequence assembly program using de Bruijn graphs. The input is a CSV file containing Sanger sequencing readings for several DNA segments. The program reads the file, cleans invalid records, converts the valid segments into JSON, constructs a de Bruijn graph using the given k-mer size, and then checks whether the graph has an Euler path. If the graph is valid, one DNA sequence is constructed and saved to an output file. The graph is also plotted as an image.

The implementation is divided into four main files. The file `ingest_data.py` contains the functions for reading and cleaning the input data and generating the JSON representation of the segments. The file `graph_builder.py` constructs and validates the de Bruijn graph and plots it. The file `dna_constructor.py` contains the functions for constructing one DNA sequence, constructing all possible DNA sequences, saving all possible sequences, and aligning segments to a DNA sequence. Finally, `project.py` contains the main workflow and the `save_output` function.

2 Implementation Details

2.1 Reading and Cleaning the Data

The `read_csv(name)` function reads the input CSV file and extracts the DNA identifier and k-mer size from the file name. The expected file name format is `DNA_x_k.csv`. A regular expression is used to extract these two values. The CSV file is then read into a pandas DataFrame, and the columns are renamed to `SegmentNr`, `Position`, `A`, `C`, `G`, and `T`.

The `clean_data(df)` function handles the expected data errors. First, exact duplicate rows are removed. Then, each segment is checked separately. A segment is removed if it has missing positions, conflicting duplicated positions, or invalid nucleotide readings where either no nucleotide is selected or more than one nucleotide is selected. After the positions are validated, duplicate segments are also removed by comparing the nucleotide pattern of each segment. The cleaned DataFrame is reset to have a clean index before being returned (becomes useful in unit tests where we compared the values of generated data with expected but index was preventing row to row comparison).

2.2 Generating Segment Sequences

The `generate_sequences(df)` function converts the cleaned DataFrame into JSON format. For each segment, the rows are sorted by `Position` to guarantee that the sequence is built in the correct order. Each row is then converted into one nucleotide depending on which of the four columns `A`, `C`, `G`, and `T` contains the value 1. The final JSON structure stores the segment number as the key and the found segment sequence as the value.

This JSON structure is simple and directly connects each segment number with its DNA sequence. It also makes the next steps easier because the graph construction only needs to iterate through the segment sequences irrespective of the segment number.

2.3 Constructing the de Bruijn Graph

The `construct_graph(json_data, k)` function first extracts all k-mers from the generated segment sequences. For each k-mer, the left node is the first $k-1$ characters and the right node is the last $k-1$ characters. An edge is added from the left node to the right node.

A `networkx.MultiDiGraph` is used instead of a normal directed graph. This is important because repeated k-mers should create repeated edges. A normal directed graph would collapse duplicate edges, which would lose information and could lead to an incorrect DNA length.

2.4 Graph Validation

The `is_valid_graph(graph)` function checks whether the graph satisfies the conditions for an Euler path. First, the graph is converted to an undirected graph and checked for connectivity using the `nx.is_connected()` built-in function. Then, the graph validation function checks the difference between the out-degree and in-degree of each node.

The graph is considered valid if all nodes are balanced, which means it has an Euler cycle. It is also valid if exactly two nodes are unbalanced, where one node has one extra outgoing edge and the other has one extra incoming edge. In all other cases, the graph is considered invalid.

2.5 Constructing DNA Sequences

The `construct_dna_sequence(graph)` function constructs one possible DNA sequence from the graph. The start node is selected as the node with $\text{out-degree} - \text{in-degree} = 1$. If no such node exists, then the graph is Eulerian and any node can be used as the start node.

The implementation uses a stack-based algorithm. An adjacency list is built from the graph edges. The algorithm follows unvisited outgoing edges until it reaches a node with no remaining outgoing edges, then backtracks and builds the path. After the path is found, the DNA sequence is reconstructed by taking the full first node and then appending the last character of every following node.

2.6 Bonus Functions

The `construct_all_dna(graph)` function constructs all possible DNA sequences by exploring all possible Euler paths. It uses a stack of states. Each state stores the current node, the set of used edges, and the path so far. When all edges have been used, the path is converted into a DNA sequence and added to a set.

The `save_all_dna(sequences, filename)` function writes all possible DNA sequences to a file, with one sequence per line.

The `align_segments(json_data, dna, filename)` function tries to align the original segments against a DNA sequence. If an alignment is possible, the aligned segments are written to the output file using dashes for spacing. If no alignment is possible, the function writes an error message and lists the segments that could not be aligned.

3 Evaluation

The main required tasks were implemented. The program can read the input CSV file, clean the data, generate segment sequences, construct the de Bruijn graph, plot the graph, check graph validity, construct one DNA sequence when possible, and save the output.

The bonus tasks were also implemented. The program constructs all possible DNA sequences and saves them in a file named using the DNA identifier. It also generates alignment files for the possible DNA sequences.

One limitation is that the alignment function uses a direct segment-overlap strategy. This works for the tested cases, but alignment can become more complicated when there are many

possible overlaps. Another limitation is that the graph plotting can become visually crowded for larger graphs.

4 Example Run

The program can be run from the command line using:

```
python project.py DNA_x_k.csv
```

For example, for the input file:

```
DNA_4_5.csv
```

4.1 Reading and Cleaning

then `read_csv` extracts the DNA identifier as 4 and the k-mer size as 5. The CSV file is read into a DataFrame with the columns:

```
SegmentNr, Position, A, C, G, T
```

After this, `clean_data` removes invalid segments, missing positions, conflicting duplicates, invalid nucleotide rows, and duplicated segments. In this example the cleaned data is the same as original (no data discrepancies). The only thing that was done is to change the column names.

Small sample of the cleaned data and the rest was included as input to `generate_sequences()` function.

```
SegmentNr,Position,A,C,G,T
1,1,0,1,0,0
1,2,0,0,0,1
1,3,1,0,0,0
...
2,12,0,0,1,0
```

4.2 Segments Generation

Then, `generate_sequences` produces a JSON structure:

```
{
  "1": "CTAATTACTCACTGG",
  "3": "CTGGGT",
  "4": "GGGTCACT",
  "2": "CACTACGCACTG"
}
```

4.3 K-mers Construction

The function `construct_graph` extracts all k-mers from these sequences. For example, if `k = 5`, the segments give:

```
CTAAT, TAATT, AATTA, ATTAC, TTACT, TACTC, ACTCA, CTCAC,
TCACT, CACTG, ACTGG, CTGGG,TGGGT, GGGTC, GGTCA, GTCAC, TCACT,
CACTA, ACTAC, CTACG, TACGC, ACGCA, CGCAC, GCACT, CACTG
```

4.4 Edges construction

These k-mers are converted into graph edges:

Left node	Right node
CTAA	TAAT
TAAT	AATT
AATT	ATTA
ATTA	TTAC
TTAC	TACT
TACT	ACTC
ACTC	CTCA
CTCA	TCAC
TCAC	CACT
CACT	ACTG
CACT	ACTG
CACT	ACTA

4.5 Plotting

The graph is then plotted and saved as:

DNA_4_5.png

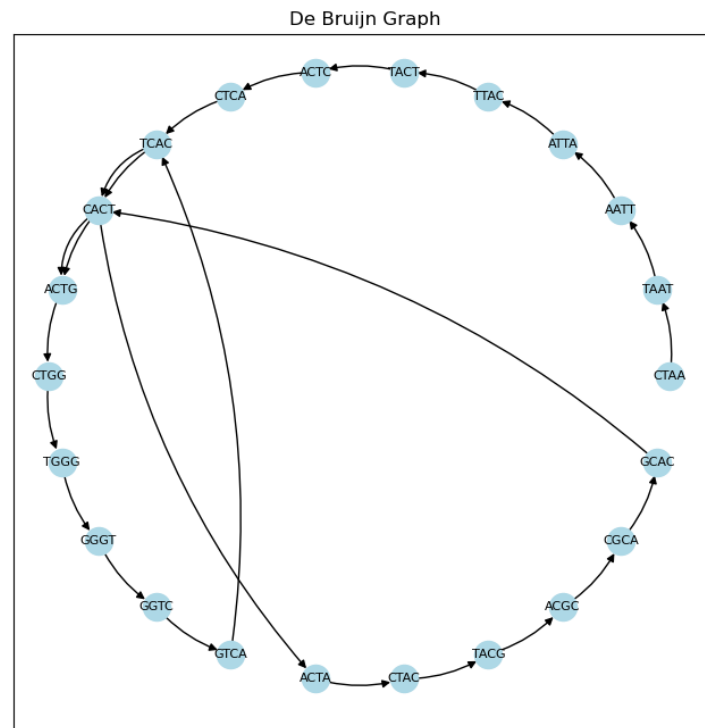


Figure 1: The constructed de Bruijn graph for DNA 4 with $k = 5$.

4.6 Sequence Construction

If the graph is valid, the DNA sequence is constructed and saved to:

DNA_4_5.txt

CTAATTACTCACTACGCACTGGGTCACTG

4.7 Bonus Output

The bonus output containing all possible DNA sequences is saved as:

DNA_4_All.txt

CTAATTACTCACTACGCACTGGGTCACTG

CTAATTACTCACTGGGTCACTACGCACTG

4.8 Sequence Alignment

Finally, the alignment files are generated using names (because we have multiple possible sequences, otherwise we only create one file called DNA_4_alignment_1.txt):

DNA_4_alignment_1.txt

For this DNA sequence

CTAATTACTCACTACGCACTGGGTCACTG

there is no possible alignment of the following segments

CTAATTACTCACTGG

CTGGGT

GGGTCACT

CACTACGCACTG

DNA_4_alignment_2.txt

CTAATTACTCACTGG-----

-----CTGGGT-----

-----GGGTCACT-----

-----CACTACGCACTG

CTAATTACTCACTGGGTCACTACGCACTG

5 Challenges

One challenge was handling repeated k-mers correctly. At first, using a normal directed graph caused repeated edges to disappear. This gave a shorter DNA sequence than expected. The solution was to use `MultiDiGraph`, which keeps multiple edges between the same pair of nodes.

Another challenge was implementing the Euler path construction without relying on built-in NetworkX Euler functions. A greedy traversal can get stuck before visiting all edges. To solve this, I used a stack-based traversal algorithm with backtracking.

A third challenge was testing exact string outputs, especially for JSON formatting and alignment files. JSON strings can look different because of spaces and indentation even when they contain the same data. Therefore, the tests compare JSON contents using `json.loads`. For alignment outputs, the exact newline and dash formatting had to be handled carefully.

6 Tests Description

The unit tests are written in `project_test.py`. In total, 21 tests were implemented.

For `clean_data`, the tests check that invalid rows and invalid segments are removed. The cases include rows with more than one nucleotide selected, duplicated positions, duplicated rows, and duplicated segments.

For `generate_sequences`, the tests check that segment sequences are correctly built from the DataFrame. Some test cases use unordered positions to confirm that sorting by position works correctly before constructing the sequence.

For `construct_graph`, the tests check that the correct edges are created from k-mers. For example, the sequence ATTACTC with $k = 5$ should produce the edges:

```
ATTA -> TTAC
TTAC -> TACT
TACT -> ACTC
```

For `is_valid_graph`, the tests include valid Euler cycles, valid Euler paths, disconnected graphs, and graphs with invalid degree imbalance. These tests are important because graph validity determines whether the DNA sequence can be constructed.

For `construct_dna_sequence`, the tests check that the function returns one of the expected valid DNA sequences from a graph.

For `construct_all_dna`, the test checks that all possible DNA sequences are returned when the graph has more than one Euler path.

For `align_segments` (I found out this was not required but I had already implemented it), the tests check both successful alignments and failed alignments. In the failed cases, the function should write the required error message and list the segments that could not be aligned.

7 Conclusion

The project was completed with the main required functions and the bonus tasks. The implementation follows the required function names and is split into separate files based on the role of each part of the program. The program can be run from the command line, produces the required graph and DNA output files, and includes unit tests for the main computational functions.